
AI AGENTS

WORKSHOP SERIES

Lora Yovcheva-Clavijo

UCLA MQE Spring 2026

SCAN THE QR CODE TO SIGN IN



Access to Materials: <https://github.com/lorayc/ai-agents>

THE WORKSHOP SERIES

01

Foundations

API calls, tool use, ReAct loop, FRED agent

02

RAG & Memory

Embeddings, vector stores, FOMC document search

03

Multi-Agent

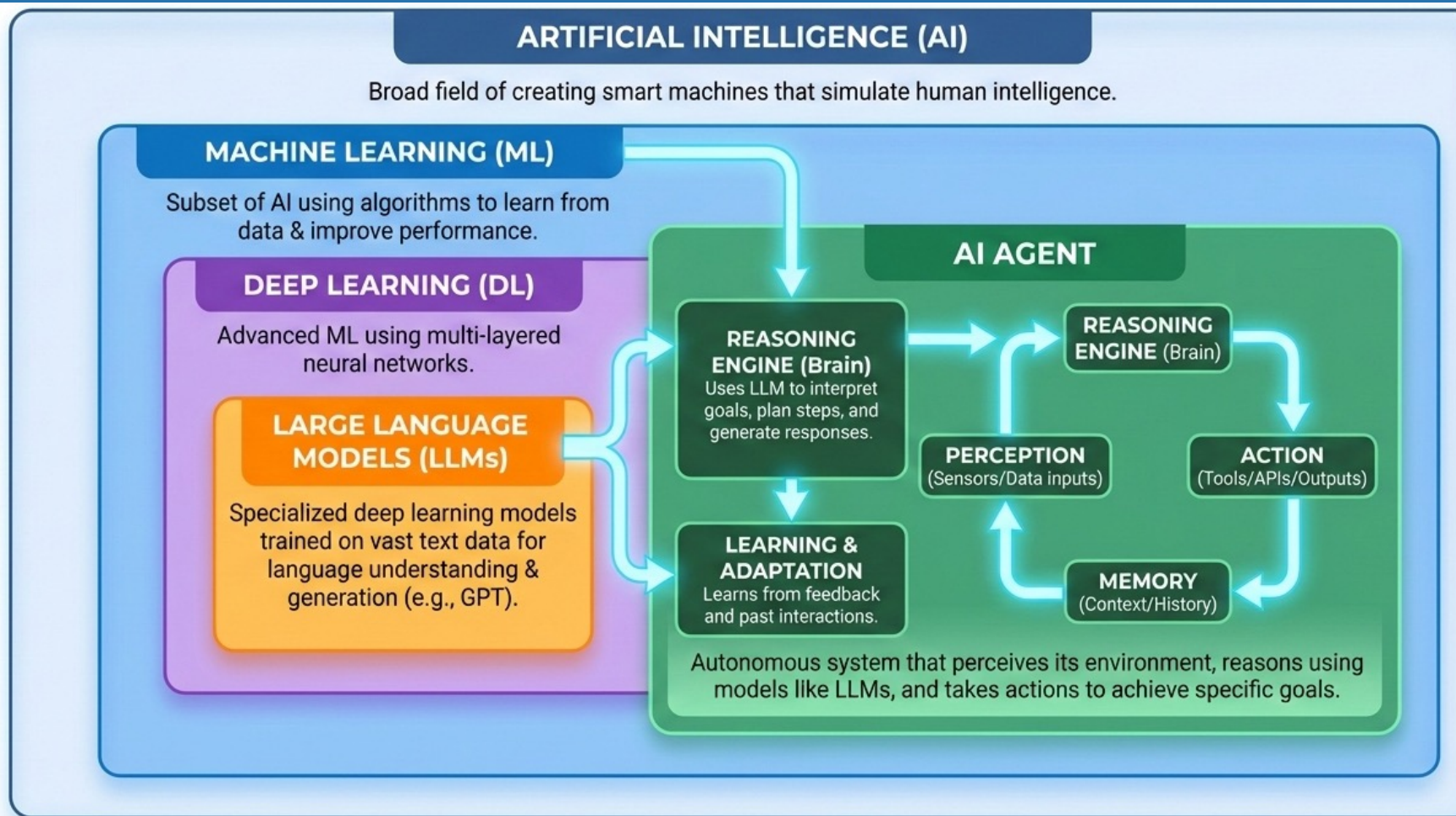
Orchestration, delegation, parallel research

04

Production & Eval

Guardrails, evaluation, deployment patterns

THE ECOSYSTEM



AI AGENT

Agent = LLM + Memory + Tools + Policy + Objective



Perception

Observe documents, APIs, databases, and external state.



Reasoning

Inference and decomposition over the gathered information.



Action

Execute tools, code, searches, calculations, API calls.



Memory

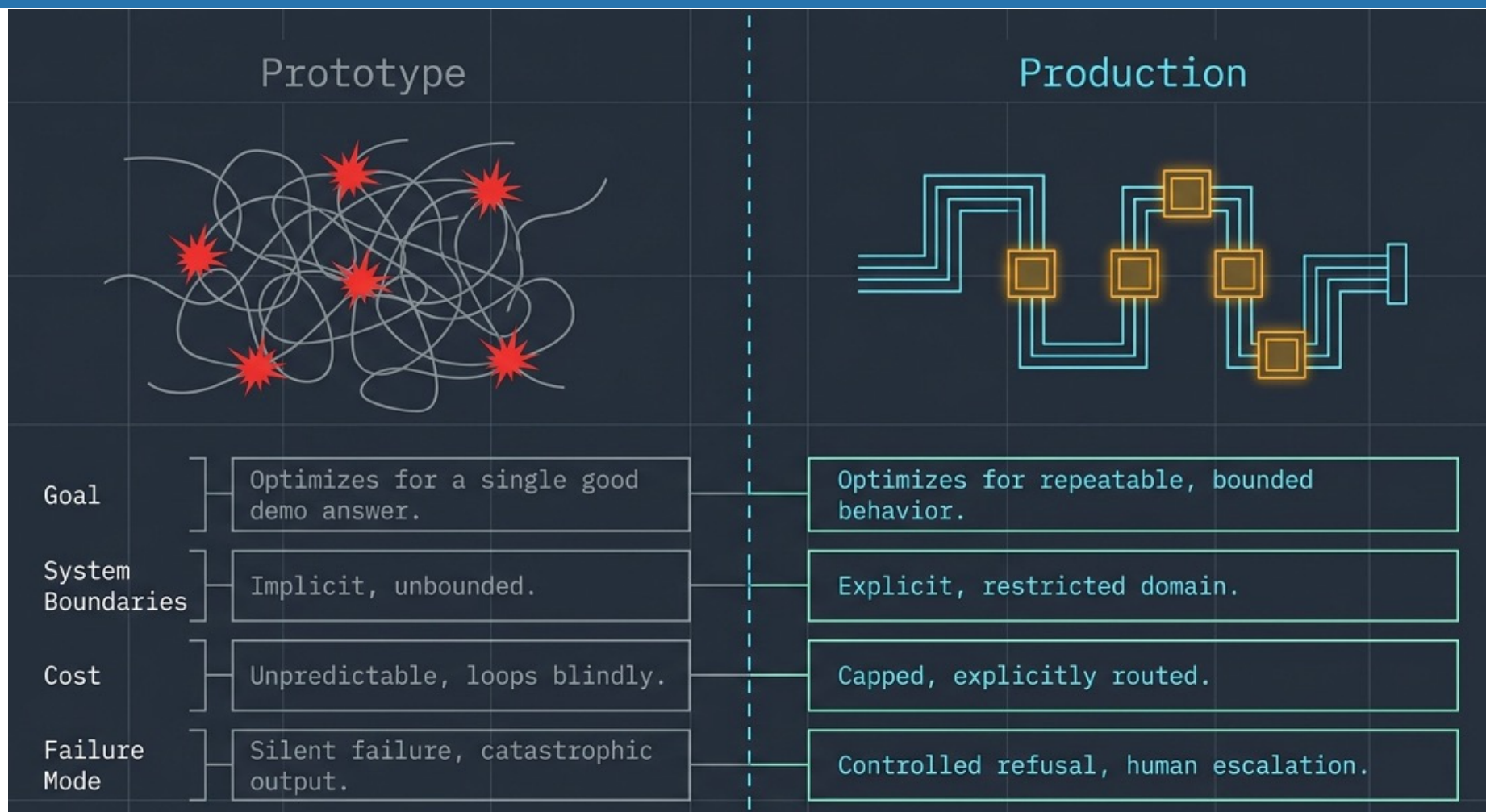
Retain intermediate state, prior outputs, long-term knowledge.



Objective

Explicit or implicit optimization target.

NEED FOR PRODUCTION



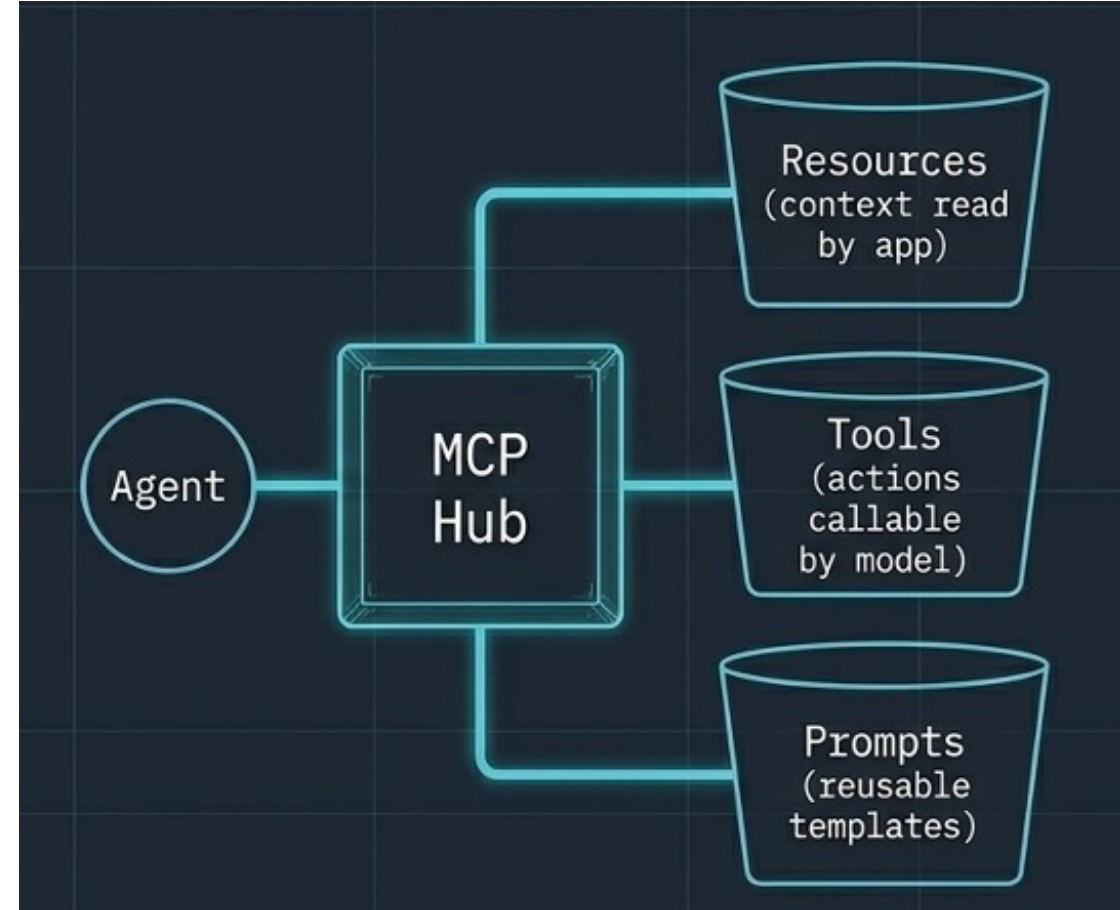
ECON COMPARISON

Econometrics	AI Agent Production
Model spec	Deployment spec
Instruments/data	Tools + MCP servers
Identification restrictions	Guardrails + permissions
Out-of-sample test	Evals + red-team cases
Audit trail	Trace log

THE PRODUCTION MAP

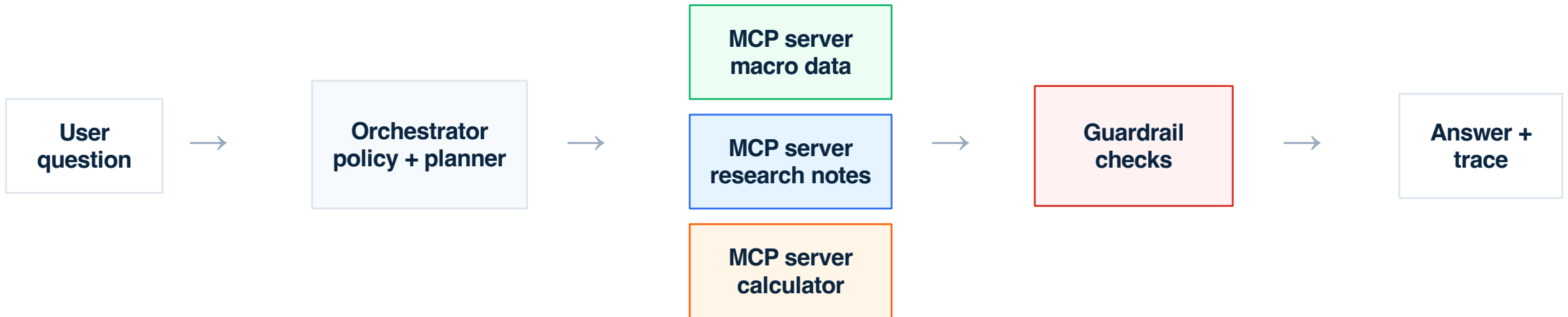


MODEL CONTEXT PROTOCOL (MCP)



ARCHITECTURE – PRODUCTION AGENT

Example: a macro research assistant that can read approved data and compute summaries, but cannot trade, email, or give personalized advice.



System boundaries

Allowed

read approved datasets; run low-risk calculations; cite retrieved evidence

Blocked

send emails; place trades; expose private files; exceed budget without review

Human review

ambiguous investment claim; policy uncertainty; high-cost synthesis; failed eval

MCP – INTERFACE STANDARD

Production design must add explicit authorization, tool allowlists, input validation, output validation, and logging.

Risk	Typical failure	Production control
Prompt injection	Tool call follows malicious text	tool-specific policies + refusal rules
Over-permissioned tools	Agent can write/delete when read-only needed	least privilege + scoped credentials
Data exfiltration	Private context appears in answer	PII filters + citation constraints
Unbounded execution	Expensive loops or repeated retries	budget caps + timeouts + circuit breaker

Security posture = protocol + policy + runtime enforcement



GUARDRAILS

Guardrails are not only refusals. They are validation layers that reduce tail risk and create auditable boundaries.

INPUT intent, scope, jailbreak, restricted domain

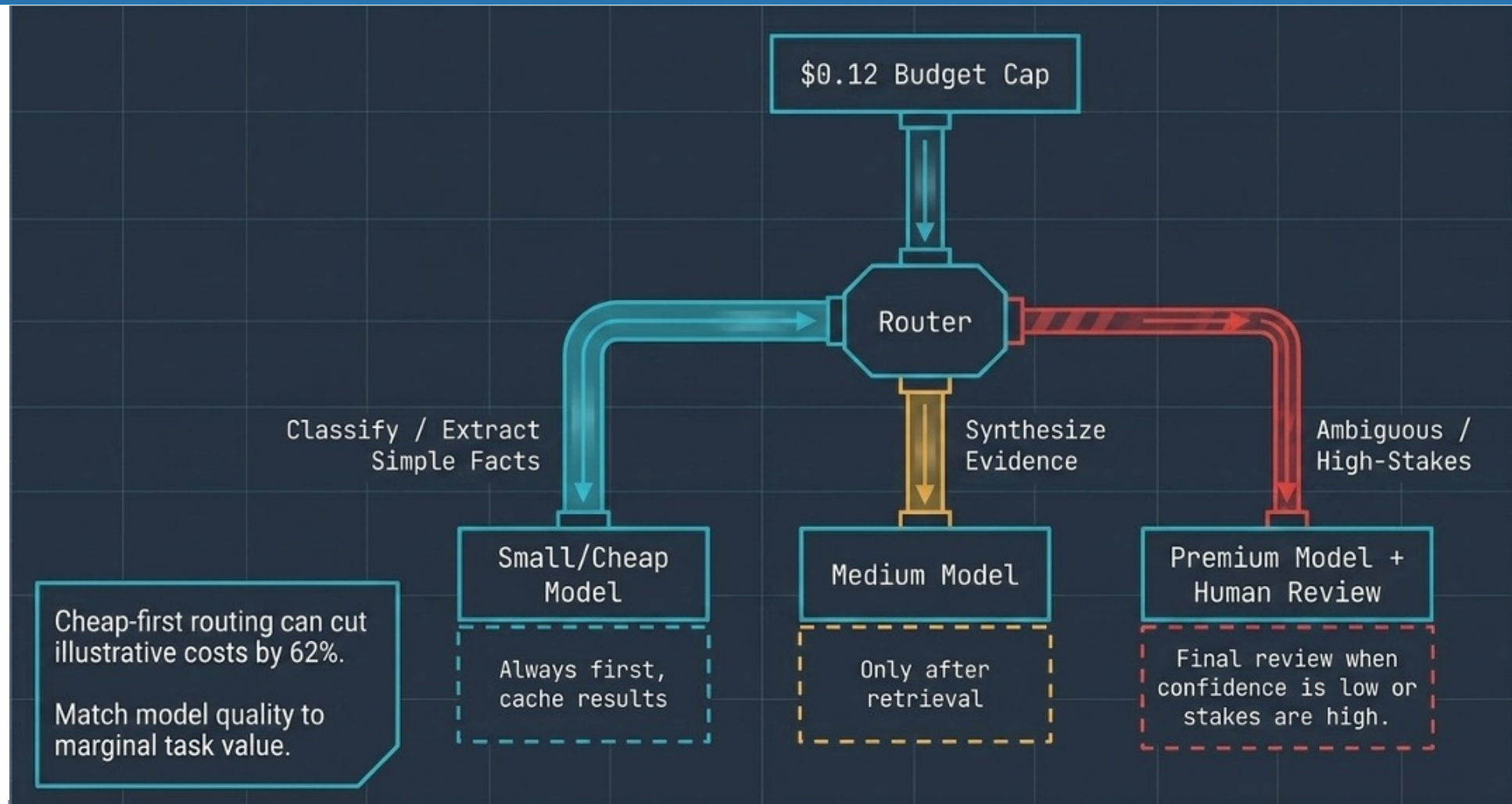
PLAN allowed tools, reasoning budget, retrieval scope

TOOL parameter validation, auth, timeout, sandbox

OUTPUT citations, uncertainty, policy compliance

**HUMAN
GATE** review when confidence is low or
stakes are high

COST CONTROL



OBSERVABILITY

A trace is the agent equivalent of a reproducible empirical workflow: inputs, tool calls, decisions, costs, errors, and final output.



Eval scorecard

87

Accuracy

PASS

91

Citations

PASS

100

Policy safety

PASS

\$0.08

Cost/run

PASS

6.2s

Latency

WATCH

FAILURE MODES

The deployment gate should catch high-probability/high-impact risks before a user does.

	Low impact	Medium impact	High impact
High likelihood	minor formatting	retrieval miss	wrong cited source
Medium likelihood	long latency	tool timeout	policy bypass
Low likelihood	rare cache miss	budget spike	unsafe action

Gate rule

Do not ship if any high-impact scenario lacks a guardrail, an owner, and a measurable test case.

DEPLOYING AN AGENT LOCALLY

1

Map

draw tools, risks, and permissions

2

MCP server

define local tool schemas

3

Spec

write versioned YAML-like contract

4

Guardrails

validate input, tool calls, output

5

Budget

track tokens and routing choice

6

Eval

pass/fail deployment gate

A runnable deployment package: agent map + MCP-style tools + spec + guardrails + cost log + eval report.